

MILESTONE 2**UI/UX Design & System Architecture**

FitPlan – Daily Activity Planner

Item	Details
Group	Group 07
Instructor	Noura Alqahtani
Milestone due	Week 6 (team target: 17 / 09 / 2026)
Grade weight	10%
GitHub repository	https://github.com/Mzoon25/FitPlan/issues

Team: Layan Alshehri, Jouri Alshurie, Lina Alshehri, Buthaynah Fareed, Bayan Alqahtani, Mzoon Alqahtani, Kadi Alqahtani

1. Goal & Overview

This document translates the FitPlan proposal (Milestone 1) into concrete screens and a technical architecture before production code is written. FitPlan takes a daily calorie goal, available time, and fitness level, then generates a daily activity plan (walking, cardio, running, stairs) that the user completes with a timer while tracking progress. All data is stored locally on the device.

Deliverables in this document

- Section 3 – Wireframes for all 8 core screens (Deliverable 1)
- Section 4 – App architecture diagram: UI / Domain / Data layers with unidirectional data flow (Deliverable 2)
- Section 5 – Navigation map showing how the Compose screens connect (Deliverable 3)
- Section 6 – Jetpack components and libraries with justification (Deliverable 4)
- Sections 7–9 – Data model, AI-use disclosure, and self-check

2. Core User Flows

Each core flow from the MVP scope maps to at least one wireframe (this answers the self-check question "Does every core user flow have a corresponding wireframe?").

#	User flow (MVP feature)	Steps	Wireframe(s)
F1	Enter goal, time, level (Goal & User Input)	Open app → enter kcal goal, minutes, level → Generate My Plan	1 Goal & Input
F2	Generate and view daily plan (Plan Generation, Activity List)	Generate → Plan screen lists activities with duration and kcal	1, 3 Daily Plan
F3	Run an activity (Activity Timer)	Plan → tap activity → Start / Pause / Finish	3, 4 Timer
F4	Complete and track progress (Completion, Progress Tracking)	Finish timer → activity marked done → progress updates	3, 2 Home, 5 Progress
F5	Review history (Local History)	History tab → list of past days with goal vs burned	6 History
F6	Set reminders (Reminder Notifications, Should)	Settings tab → toggle reminders and choose time	8 Reminder Settings
F7	See weekly summary (Weekly Statistics, Could)	History → Weekly Statistics	7 Weekly Statistics

3. Wireframes (Core Screens)

Low-fidelity wireframes in a Material 3 style, designed as a 360 × 720 dp phone layout. The green top bar and bottom navigation (Home, Plan, History, Settings) are shared across screens. Screens 4, 5, and 7 are secondary destinations with a back arrow.

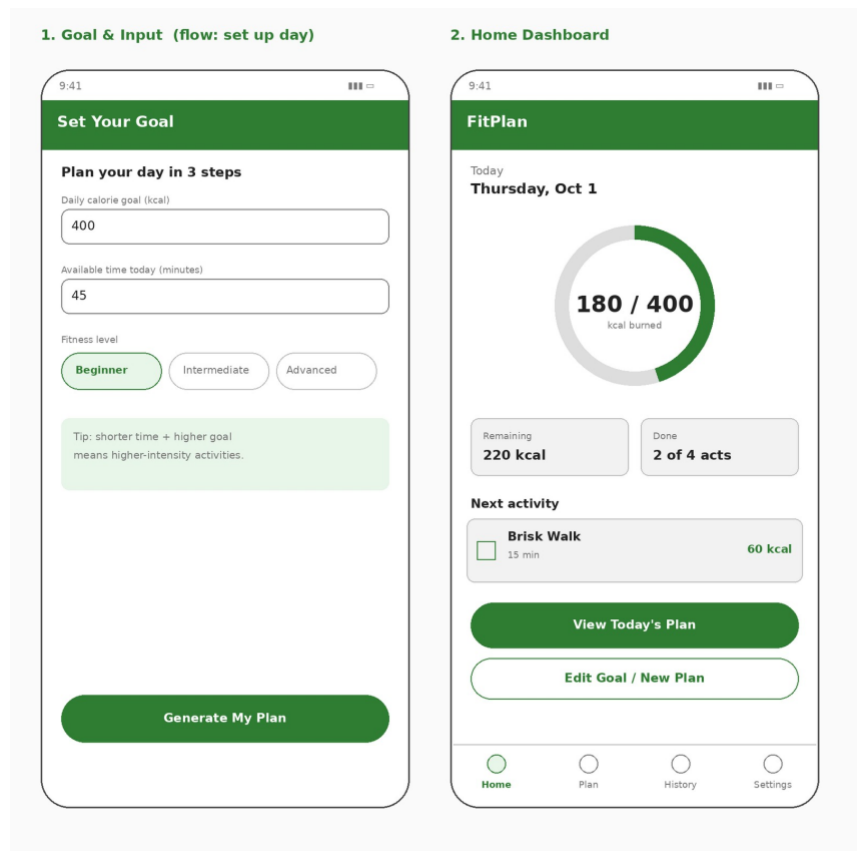


Figure 1 – Screens 1 (Goal & Input) and 2 (Home Dashboard)

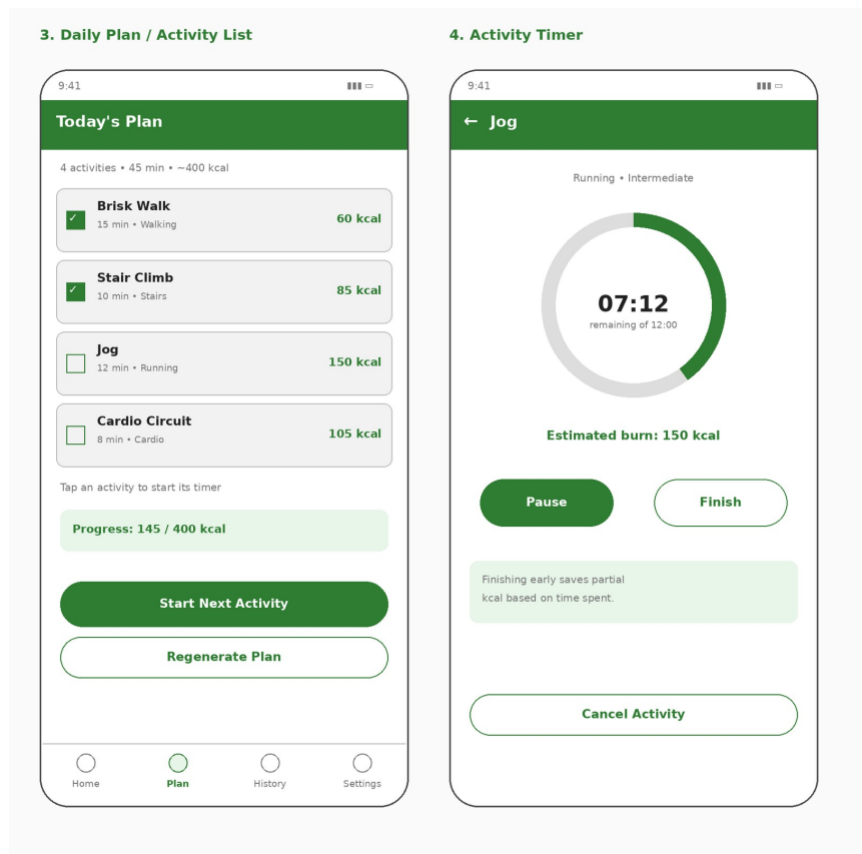


Figure 2 – Screens 3 (Daily Plan / Activity List) and 4 (Activity Timer)

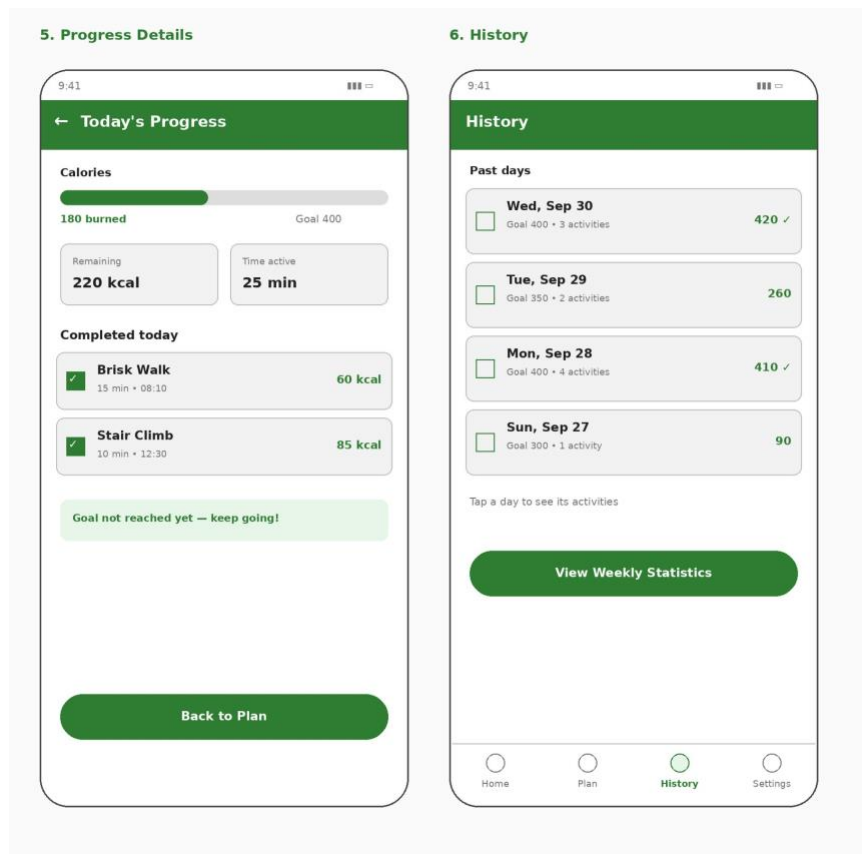


Figure 3 – Screens 5 (Progress Details) and 6 (History)

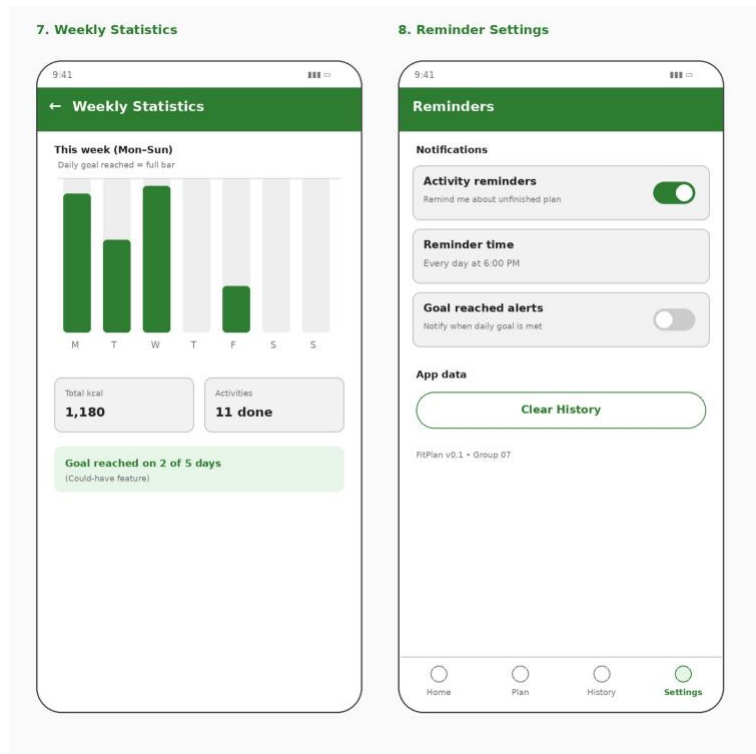


Figure 4 – Screens 7 (Weekly Statistics) and 8 (Reminder Settings)

Screen summary

#	Screen	Purpose / key elements	Priority
1	Goal & Input	Calorie goal field, available time field, fitness level chips, Generate button	Must
2	Home Dashboard	Progress ring (burned vs goal), remaining kcal, activities done, next activity, shortcuts	Must
3	Daily Plan	Activity list with duration and estimated kcal, check state, start next, regenerate	Must
4	Activity Timer	Countdown, Pause, Finish, estimated burn, cancel	Must
5	Progress Details	Calories bar, remaining, active time, completed list	Must
6	History	List of past days: goal, activities, kcal burned	Must
7	Weekly Statistics	7-day bar chart, totals, goals reached	Could
8	Reminder Settings	Reminder toggle, time, goal alert, clear history	Should

4. App Architecture

FitPlan follows the recommended Android layered architecture with three layers: UI, Domain, and Data. State flows upward and user events flow downward (unidirectional data flow, UDF).

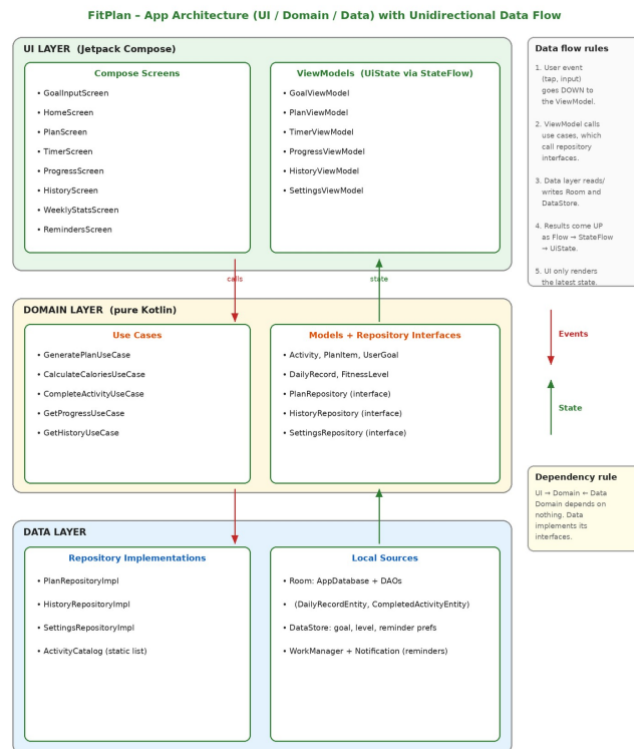


Figure 5 – Architecture diagram: UI / Domain / Data layers and unidirectional data flow

Layer responsibilities

Layer	Responsibility
UI	Compose screens render a single immutable UiState per screen and send user events to a ViewModel. Screens hold no business logic. The TimerViewModel owns the countdown using a coroutine ticker.
Domain	Pure Kotlin business rules: GeneratePlanUseCase selects activities that fit the goal, time, and level; CalculateCaloriesUseCase computes $kcal = rate(activity, level) \times minutes$; CompleteActivityUseCase and GetProgressUseCase update and read daily progress. Defines repository interfaces.
Data	Repository implementations map between Room entities / DataStore values and domain models. Room stores daily records and completed activities; DataStore stores goal, level, and reminder preferences; WorkManager schedules reminder notifications.

Example flow: finishing an activity

1. User taps Finish on the Timer screen (event).
2. TimerViewModel calls CompleteActivityUseCase.
3. The use case calls HistoryRepository, which writes to Room.
4. Room emits an updated Flow; ProgressViewModel exposes it as StateFlow<ProgressUiState>.
5. Home, Plan, and Progress screens recompose automatically with the new totals.

5. Navigation Map

Navigation Compose manages a single NavHost. Home, Plan, History, and Settings are top-level destinations in the bottom navigation bar. On first launch (no saved goal) the app starts at Goal & Input and, after Generate Plan, moves to the Plan tab.

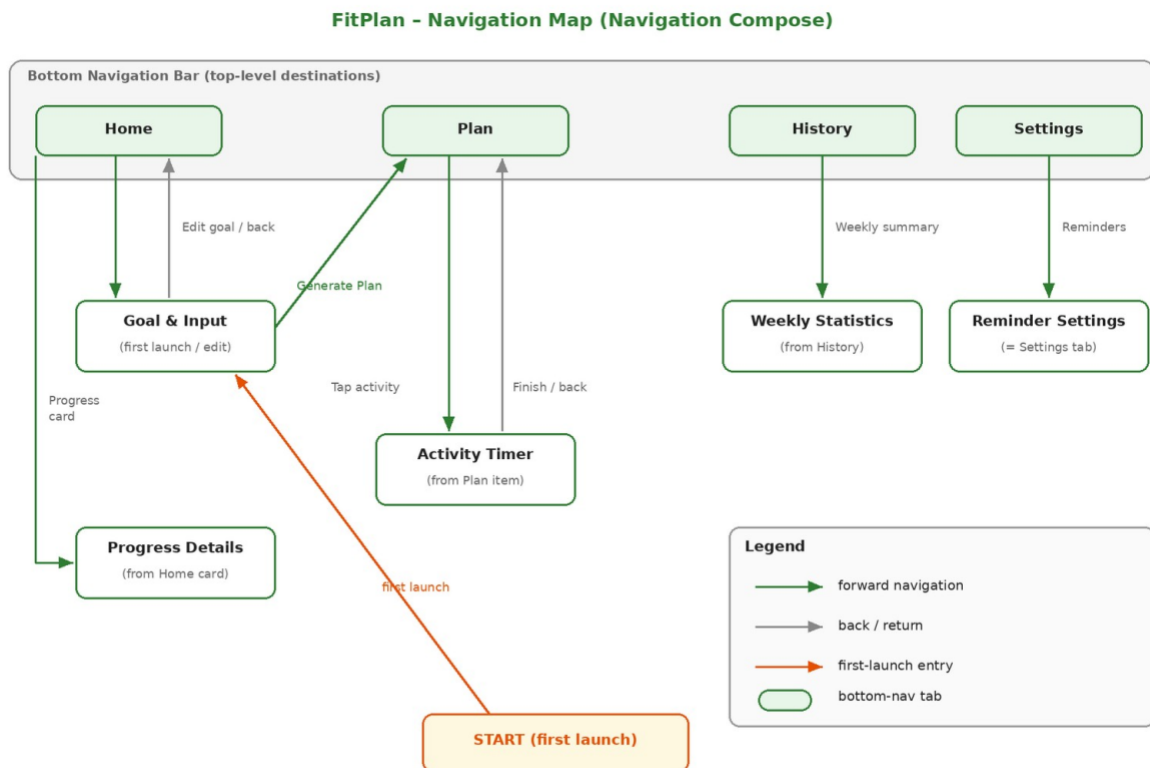


Figure 6 – Navigation map of Compose screens

From	To	Trigger
START	Goal & Input	First launch with no saved goal
Goal & Input	Plan	Tap Generate My Plan
Home	Goal & Input	Tap Edit Goal / New Plan
Home	Progress Details	Tap progress ring / card
Plan	Activity Timer	Tap an activity or Start Next Activity
Activity Timer	Plan	Finish, cancel, or back
History	Weekly Statistics	Tap View Weekly Statistics
Settings tab	Reminder Settings	Bottom navigation (same screen)

6. Jetpack Components & Libraries

Component / library	Used for	Justification
Jetpack Compose + Material 3	All screens, inputs, cards, progress indicators	Required by the course; declarative UI is easy to match to state and supports the reusable list and progress components in our wireframes.
Navigation Compose	Navigation map and bottom nav	Official navigation solution for Compose; handles back stack and bottom-bar tabs with save/restore state.
ViewModel (lifecycle-viewmodel-compose)	Screen state holders	Keeps state through configuration changes and hosts the timer so it does not reset on rotation.
Kotlin Coroutines + Flow / StateFlow	Async work, timer ticks, reactive state	Enables unidirectional data flow: Room Flows feed StateFlow to the UI. collectAsStateWithLifecycle avoids wasted updates.
Room	Local history of daily goals and completed activities	Structured local storage with SQL queries (e.g., last 7 days) and Flow support. Meets the local-storage requirement.
DataStore (Preferences)	Calorie goal, level, reminder settings	Simple key-value settings with coroutine support; safer than SharedPreferences.
WorkManager + NotificationCompat	Optional reminders	Reliable scheduling that survives restarts and respects battery limits; needs POST_NOTIFICATIONS permission on Android 13+.
Compose Canvas (no extra chart library)	Progress ring and weekly bar chart	Our charts are simple; avoiding a third-party library reduces dependencies and risk.
JUnit, Compose UI Test	Testing use cases and key screens	Domain logic is pure Kotlin so plan and calorie rules can be unit tested; supports the QA role.

Dependency injection: for the beta we plan manual constructor injection (an AppContainer class) to keep the learning curve low; Hilt can be added later if time allows.

7. Data Model (Room / DataStore)

Entity	Fields	Notes
DailyRecordEntity	date (PK), goalKcal, availableMinutes, fitnessLevel	One row per day
CompletedActivityEntity	id (PK), date (FK), activityName, minutes, kcal, completedAt	Many per day
PlanItem (domain model)	activityName, type, minutes, estimatedKcal, isDone	Generated each day
DataStore keys	last goal, last level, remindersEnabled, reminderTime	User preferences

8. AI Use Disclosure

Per the course AI-Use Policy, AI was used only as a design aid ("Pair Programmer"). No production code was generated and AI is not a feature of the app.

Tool / Model	Where used	Role	Reviewed by
Claude (Anthropic)	Draft of wireframes, architecture diagram, navigation map, and Jetpack components list (Milestone 2 design artifacts)	Pair Programmer (design aid)	All team members (Group 07)

Team note: all members should review and understand every artifact before submission, and update this table if the design is changed or other tools are used.

9. Self-Check Before Submitting

- ☒ Does the architecture diagram clearly separate UI, Domain, and Data layers? Yes – Figure 5.
- ☒ Have we noted anywhere we used AI assistance while producing these design artifacts? Yes – Section 8.
- ☒ Does every core user flow have a corresponding wireframe? Yes – Section 2 table (F1–F7).

GitHub evidence checklist

- Commit this file and the image sources to /docs (e.g., docs/milestone2/...).
- Close all Milestone 2 issues on the Project board.
- Add the repository URL to the proposal table.